

# AFRILANG Stdlib

## std/timezone

**Français.** Bibliothèque AFRILANG 'std/timezone' (niveau simple, catégorie time). Elle expose 2 fonction(s) : utcOffsetHours, localHour.

```
'import "std/timezone"' · 'use timezone'
```

```
- 'utcOffsetHours(ignored number) → number' - 'localHour(utcHour  
number) → number'
```

**English.** AFRILANG library 'std/timezone' (tier simple, category time). Exports 2 function(s): utcOffsetHours, localHour.

```
'import "std/timezone"' · 'use timezone'
```

```
- 'utcOffsetHours(ignored number) → number' - 'localHour(utcHour  
number) → number'
```

|                       |               |               |
|-----------------------|---------------|---------------|
| Catégorie / Category  | Temps & dates |               |
| Niveau / Tier         | simple        | June 16, 2026 |
| Fonctions / Functions | 2             |               |

## Contents

## Français

### 1. Vue d'ensemble

Bibliothèque AFRILANG 'std/timezone' (niveau simple, catégorie time). Elle expose 2 fonction(s) : utcOffsetHours, localHour.

'import "std/timezone"' · 'use timezone'

```
- `utcOffsetHours(ignored number) → number`
- `localHour(utcHour number) → number`
```

### 2. Installation & import

Ajoutez le module à votre programme :

```
import "std/timezone"
use timezone
```

Niveau : simple · Catégorie : Temps & dates  
Horloge, calendrier, délais, formatage temporel.

### 3. Référence API

- utcOffsetHours(ignored number) → number•  
localHour(utcHour number) → number

### 4. Exemples d'utilisation

```
import "std/timezone"
use timezone
```

```
create result = utcOffsetHours(/* args */)
say result
```

```
create result = localHour(/* args */)
say result
```

### 5. Bonnes pratiques

- Préférez `import "std/timezone"` en tête de fichier. •  
Lancez `afrilang check` avant de livrer. •  
Combinez avec les modules stdlib de la même catégorie. •  
Voir /stdlib/ pour le source live et les démos playground. •  
Signalez les problèmes sur GitHub si une export manque.

### 6. Annexe — source

module timezone

```
function utcOffsetHours(ignored number) returns number
end
```

```
function localHour(utcHour number) returns number
end
end
```

## English

### 1. Overview

AFRILANG library 'std/timezone' (tier simple, category time). Exports 2 function(s): `utcOffsetHours`, `localHour`.

'import "std/timezone"' · 'use timezone'

```
- `utcOffsetHours(ignored number) → number`
- `localHour(utcHour number) → number`
```

### 2. Installation & import

Add the module to your program:

```
import "std/timezone"
use timezone
```

Tier: simple · Category: Time & dates  
Clock, calendar, delays, time formatting.

### 3. API reference

- `utcOffsetHours(ignored number) → number`•  
`localHour(utcHour number) → number`

### 4. Usage examples

```
import "std/timezone"
use timezone
```

```
create result = utcOffsetHours(/* args */)
say result
```

```
create result = localHour(/* args */)
say result
```

### 5. Best practices

- Prefer ``import "std/timezone"`` at the top of your file. •  
Run ``afriLang check`` before shipping. •  
Combine with related stdlib modules from the same category. •  
See `/stdlib/` for the live source and playground demos. •  
Report issues on GitHub if an export is missing or undocumented.

### 6. Appendix — source

module timezone

```
function utcOffsetHours(ignored number) returns number
end
```

```
function localHour(utcHour number) returns number
end
end
```

## Référence rapide / Quick reference

- Import : `import "std/timezone"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/timezone/>

## Source (extrait)

```
module timezone

function utcOffsetHours(ignored number) returns number
end

function localHour(utcHour number) returns number
end
end
```